ForecastLabAI — overview and tech stack

Problem

Explain in one breath what ForecastLabAI is and what it is built with, without overclaiming.

Instant solution

Pitch: ForecastLabAI is not just a model. It is an end-to-end retail demand-forecasting app: data ingestion → features → training → backtesting → model selection/registry → forecasts and what-if scenarios → AI decision support (RAG assistant + PydanticAI experiment agent).

Area	Technologies (verified in repo)
Language	Python ≥ 3.12 (`pyproject.toml`)
API	FastAPI, Pydantic v2, Uvicorn
Database	PostgreSQL 16 + pgvector (`pgvector/pgvector:pg16`)
DB access	SQLAlchemy 2 (async), asyncpg, Alembic
Data	Pandas, NumPy
ML	scikit-learn; LightGBM, XGBoost as optional extras (`ml-lightgbm`, `ml-xgboost`)
GenAI	pydantic-ai, openai, anthropic, tiktoken
RAG	embeddings + pgvector (HNSW, cosine)
Frontend	React 19, TypeScript, Vite 7, Tailwind CSS 4, shadcn/ui, Recharts, pnpm
Infra	Docker, Docker Compose; Ollama optional (`gpu` profile)
Tooling	uv, pytest, Ruff, mypy, Pyright, GitHub Actions, release-please

Frontend pages: Data Explorer, Forecasts, Demand Planner, What-If Planner, Knowledge, Agent Chat/Guide, AI Models Console (`/admin` , swaps LLM/embedding model live).

Why it works / gotchas

- Say "optional" for LightGBM, XGBoost, Random Forest and Ollama — they are extras or feature flags, off by default.
- Agent models use `provider:model` strings; default `anthropic:claude-sonnet-4-5` , fallback `openai:gpt-4o` (`app/core/config.py`).
- Three levels to move between in the interview: business problem → implementation → engineering decision.

See also

- Architecture · ML workflow · Models
- Feature leakage · Backtesting
- RAG · Agents · DevOps
- Interview Q&A · 2-minute pitch

Diagram brief

- Type: layered stack (top → bottom).
- Layers: Frontend (React/TS/Vite, Tailwind, shadcn, Recharts) → API (FastAPI, Pydantic) → Domain (ML: sklearn/LightGBM/XGBoost · GenAI: PydanticAI, OpenAI, Anthropic) → Data (PostgreSQL 16 + pgvector via SQLAlchemy/asyncpg/Alembic).
- Side rail: Tooling (uv, pytest, Ruff, mypy, Pyright, GitHub Actions) and Runtime (Docker Compose, optional Ollama).
- Mark optional parts with dashed borders. Do NOT show Kubernetes, cloud services or auth.